Lab 3

$

Debugging: -fsanitize=address

Add this flag to your Makefile:

```
-fsanitize=address
```

If your program crashes due to accessing memory (nearly guaranteed when dealing with pointers) it will print the line it happened.

Memory & pointers -- visual reference

https://github.com/storchdev/cs-materials/blob/80a33f68eb4b9f1640eb1ed32bcbd2dd4ac8343b/c/memory-c-programmer.pdf

My visual memory slides! (warning: very long). Already battle tested but mistakes are possible - please let me know.

Explains memory and pointers visually, in depth.

Parts 1, 3, 4 – directly relevant to this lab.

Part 2 – helpful if you're still confused after part 1.

Strings are char arrays

A C string is just an array of `char`.

```
char word[6] = "hello";
```

Why we need 6 chars to store a 5-character string? There is a hidden null byte “\0” at the end. This way, programs know how long strings are without needing to store their length.

The null terminator

Every C string ends with a `\0` byte (value 0) marking the end.

`"hello"` is 5 letters but takes **6** bytes: `h e l l o \0`.

word	
0x40	h
0x44	e
0x48	l
0x4C	l
0x50	o
0x54	\0

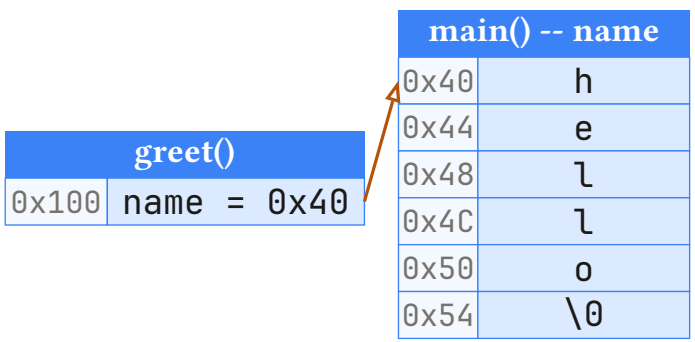
An n-character string needs **n + 1** bytes.

Arrays decay to pointers

Passing an array to a function passes a pointer to its first element – not a copy of the array.

```
void greet(char *name) { ... }

char name[6] = "hello";
greet(name);   // decays to &name[0]
```



The `greet` function ONLY gets the pointer to the start. You can only figure out the length inside `greet` if you loop thru the whole string until you hit the null char.

Questions?

Office hours (Hudson)

Monday, 12:15 PM to 2:15 PM @ ITE 214

Discord

Ask any time

I will try to be quick!

Credits:

Slides made with Typst (typst.app) w/ help from Gen. AI